Big Data Analytics

MINI-PROJECT REPORT

ON

"GitPulse: Big Data Analytics on Open Source GitHub Activity"

BY

Ayaan Memon (A-759),
Tanmay Meshram (A-761),
Kshitij Desai (A-772)

Under the guidance of

Internal Guide
Prof. Bhavesh Panchal

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
Juhu-Versova Link Road, Versova, Andheri (W), Mumbai-53
Department of Computer Engineering
University of Mumbai

October 2026

Declaration

We wish to state that the work embodied in this project titled **"GitPulse: Big Data Analytics on Open Source GitHub Activity"** forms our own contribution to the work carried out under the guidance of **"Prof. Bhavesh Panchal"** at the Rajiv Gandhi Institute of Technology.

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Students' Signatures)

1. Ayaan Memon (A-759)

2. Tanmay Meshram (A-761)

3. Kshitij Desai (A-772)

Abstract

GitPulse is an end-to-end big data analytics platform that studies public activity on GitHub, the largest open source platform in the world. It ingests the hourly event dumps published by GH Archive, about 167,000 events per hour, together with repository metadata from the GitHub REST API, and answers ten analytical questions about trending repositories, language trends, activity patterns, bot activity, maintainer health, team structure, repository recommendations, viral-repository prediction, live spike detection and repository clustering.

The system uses twelve tools from the Hadoop ecosystem on a Dockerised multi-node cluster with two DataNodes (replication factor 2) and two YARN NodeManagers. Apache Pig with a Java UDF cleans and deduplicates the raw JSON and tags bot accounts. A custom MapReduce job ranks the top repositories per day. Apache Hive builds curated tables and enforces data-quality gates that stop the pipeline on any violation. Apache Sqoop moves data between PostgreSQL and Hadoop. Apache Spark and Apache Mahout build features, a viral-repository classifier, item-based recommendations and k-means clusters. Apache HBase serves results with salted row keys, Apache Impala answers dashboard queries in about one second, and Apache Kafka with Apache Storm processes a replayed event stream in real time. Apache Airflow orchestrates both pipelines, and memory-aware stages let the full stack run on one laptop.

On 2.4 million real events the pipeline accounted for every record, with no malformed lines and one duplicate removed. Bots produced 28% to 32% of all events and more than half of all comments. Mahout item-based collaborative filtering reached a recall at 10 of 0.251, about six times the popularity baseline, and the viral model reached an AUC-ROC of 0.73. The streaming path processed a full GitHub hour at 60 times speed with zero failed tuples. A benchmark of five engines showed Impala answering in about 1.3 seconds against 20 to 170 seconds for the YARN-based engines. An end-to-end test run on a Windows laptop passed all 25 pipeline steps and all tests.

Keywords: Big Data, Hadoop, HDFS, MapReduce, Pig, Hive, Spark, Mahout, HBase, Kafka, Storm, Impala, Airflow, GitHub, GH Archive.

Contents

Sr. No.	Contents	Pg. No.
	Declaration	i
	Abstract	ii
	List of Figures	iv
	List of Tables	v
1	**Introduction**	1
	1.1 Introduction Description	1
	1.2 Ambiguity in Project	1
	1.3 Objectives	2
2	**Proposed System**	3
	2.1 Proposed Statement	3
	2.2 Design Details	3
	2.3 Data Pre-Processing Steps	5
	2.4 Main Processing Steps	6
	2.5 Tools Used	8
	2.6 Software and Hardware Requirements	9
	2.7 Dataset Used	10
3	**Results**	11
	3.1 Results	11
4	**Conclusion and future work**	20
	References	21

List of Figures

Figures	Pg. No.
Figure 2.1: Overall architecture of GitPulse	4
Figure 2.2: HDFS and YARN cluster layout	4
Figure 2.3: Daily batch pipeline	5
Figure 2.4: Real-time path: Kafka to Storm to HBase	8
Figure 2.5: The two Airflow DAGs	8
Figure 3.1: Data-quality funnel	11
Figure 3.2: Share of events created by bots	12
Figure 3.3: Active repositories by team type	13
Figure 3.4: Comparison of recommendation methods	13
Figure 3.5: Median query time by engine, medium dataset	15
Figure 3.6: Slowest steps of the test run	16
Figure 3.7: HDFS NameNode: two DataNodes in service	17
Figure 3.8: MapReduce JobHistory: completed jobs	18
Figure 3.9: Impala daemon: completed queries	18
Figure 3.10: HBase Master with its region server	19

List of Tables

Tables	Pg. No.
Table 1.1: Analytical questions and the tools that answer them	2
Table 2.1: Memory-aware stages	5
Table 2.2: Tools used and their roles	8
Table 2.3: Software and hardware requirements	9
Table 2.4: Hours of GH Archive data used	10
Table 2.5: Dataset metrics	10
Table 3.1: Record counts through the pipeline	11
Table 3.2: Recommendation results	13
Table 3.3: Viral prediction results	14
Table 3.4: Median query time in seconds by engine	14
Table 3.5: Steps of the end-to-end test run	15
Table 3.6: Summary of tests	16

CHAPTER 1

Introduction

1.1 Introduction Description

GitHub is the largest home of open source software in the world. Every action a developer takes there, such as pushing code, starring a repository, opening a pull request, filing an issue or writing a comment, is recorded as a public event. The GH Archive project collects all of these events and publishes them as one compressed JSON file for every hour of every day. A single hour holds about 167,000 events and a single day holds about four million, which makes this data too large and too fast-moving to study with a spreadsheet or a single database query.

GitPulse is our end-to-end big data analytics platform for this data. It ingests the hourly GH Archive dumps together with repository details from the GitHub REST API, cleans and checks them, stores them on a Hadoop cluster and answers ten analytical questions: which repositories are trending, which languages are growing, how much of the activity comes from bots, how healthy maintainers are, which repositories a developer is likely to enjoy, whether a new repository will go viral, and what is happening right now in a live stream of events.

The main idea of the project is that **every tool in the Hadoop ecosystem does the job it suits best**. Pig cleans raw JSON, a hand-written MapReduce job ranks repositories, Hive and Impala answer SQL questions, Spark and Mahout build machine learning models, HBase serves millisecond lookups, Kafka and Storm handle the real-time stream, Sqoop moves data in and out of PostgreSQL, and Airflow runs everything in the right order. All twelve tools run on a multi-node cluster built with Docker Compose on a single laptop.

1.2 Ambiguity in Project

Working with real GitHub data raised several ambiguities that the design had to resolve:

- **Bot or human?** Close to a third of all events are created by automated accounts such as dependabot and CI tools. A repository with thousands of automated pushes is not really trending. GitPulse tags bots in three ways (a [bot] suffix, a known-bots list and a -bot name rule) and ranks repositories by distinct human users.
- **What does trending mean?** Raw push counts are led by automated push-spam repositories, many of which are deleted within days. Stars from humans are a much better signal of interest, so the trending tables use human stars and distinct actors.

- **Deleted repositories.** Some repositories that appear in the events no longer exist when the GitHub API is called. The catalogue records these with their HTTP status instead of failing.
- **Late events in the stream.** In a live stream, events can arrive out of order. Storm windows use the event time inside each record with a 10 second lag, so an event is counted in the minute it happened, not the minute it arrived.
- **Changing data format.** In October 2025 GitHub trimmed the payload of public events. GitPulse uses September 2025 data and the quality gate counts any events already in the trimmed format.
- **Time zones.** GH Archive uses UTC. Activity patterns are reported in both UTC and Indian Standard Time (UTC + 5:30).
- **Limited hardware.** A real Hadoop cluster has many machines, but this project runs on one laptop. Memory-aware stages start only the services the current step needs.

1.3 Objectives

The project answers the ten questions in Table 1.1. Each question is matched with the tools that suit it best.

Table 1.1: Analytical questions and the tools that answer them

#	Analytical question	Tools used
Q1	Most active and fastest-rising repositories per day and week	MapReduce (top-N), Hive, Sqoop export
Q2	Language and topic trends	Sqoop catalogue, Hive join, Impala
Q3	Activity by hour and weekday (UTC and IST)	Hive, Impala
Q4	Bot versus human share of activity	Pig (bot tagging), Hive
Q5	Maintainer health: pull request merge time, issue close ratio	Hive
Q6	Fork to star ratio, solo versus community repositories	Spark (Gini, top contributor share)
Q7	"Starred X also starred Y" recommendations	Mahout item CF, Spark ALS, HBase
Q8	Will a new repository go viral?	Spark MLlib (logistic regression, GBT)
Q9	Live events per minute, top repositories, spike alerts	Kafka, Storm, HBase
Q10	Clusters of repositories with similar activity	Mahout k-means

CHAPTER 2

Proposed System

2.1 Proposed Statement

The proposed system is a complete batch and real-time analytics pipeline for public GitHub activity that runs on commodity hardware. It follows the Lambda architecture. A batch layer on HDFS and YARN produces accurate, quality-checked tables for each day of data, and a speed layer built on Kafka and Storm produces live counters and alerts within seconds. A serving layer of Impala, HBase and PostgreSQL makes both sets of results available to a Tableau dashboard and to command-line lookups.

Every pipeline step is exposed as one command of a single entry point, bin/pulse, which runs inside an edge container that holds all the client tools. The same command is used by the Makefile, by the Airflow DAGs and during a live demonstration, so the system behaves the same way no matter how it is started. Each step overwrites only its own day or partition, so any step can be retried safely, and data-quality gates stop the pipeline as soon as the numbers do not add up.

2.2 Design Details

Architecture / Flow Diagram

Figure 2.1 shows the overall architecture. Data enters on the left from GH Archive and the GitHub REST API. Inside the HDFS and YARN cluster it moves through four zones: raw JSON, clean Parquet, curated Hive tables and the machine learning zone. Results are served by Impala, HBase and PostgreSQL, and the speed layer at the bottom carries a replayed event stream through Kafka and Storm into HBase. Apache Airflow orchestrates the batch steps.

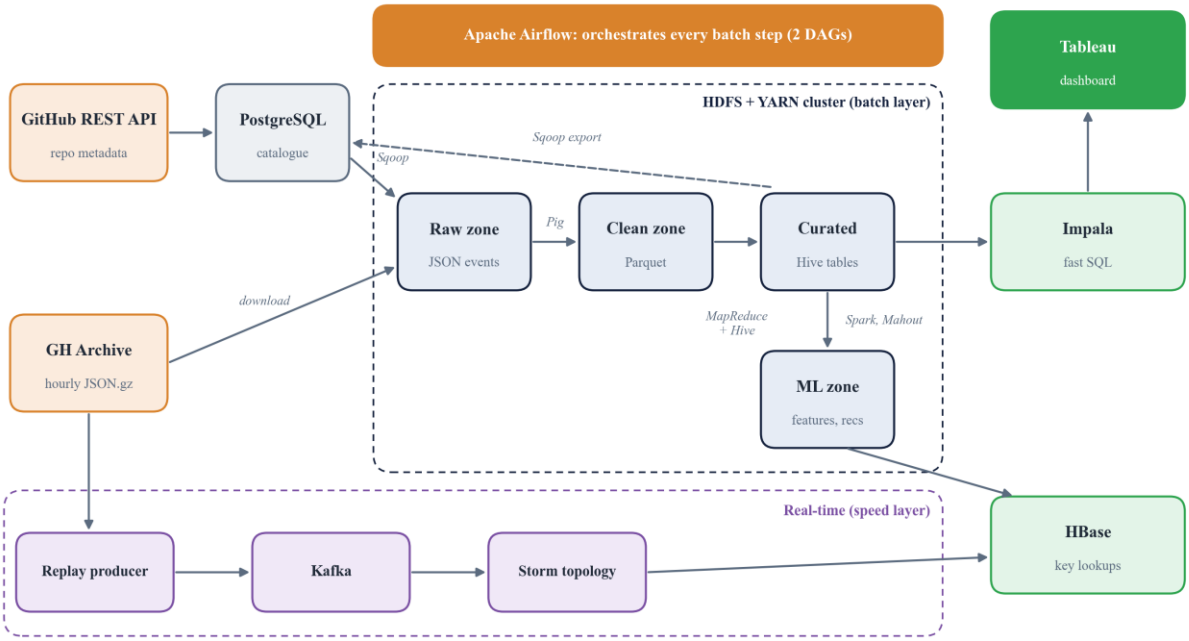


Figure 2.1: Overall architecture of GitPulse

Cluster Design

The Hadoop cluster is made of separate Docker containers that play the roles of separate machines (Figure 2.2). HDFS has one NameNode and two DataNodes with a replication factor of 2, so every block of every file is stored on both DataNodes. YARN has one ResourceManager and two NodeManagers, and a JobHistory server keeps the logs and counters of finished jobs. In total the project defines 23 services.

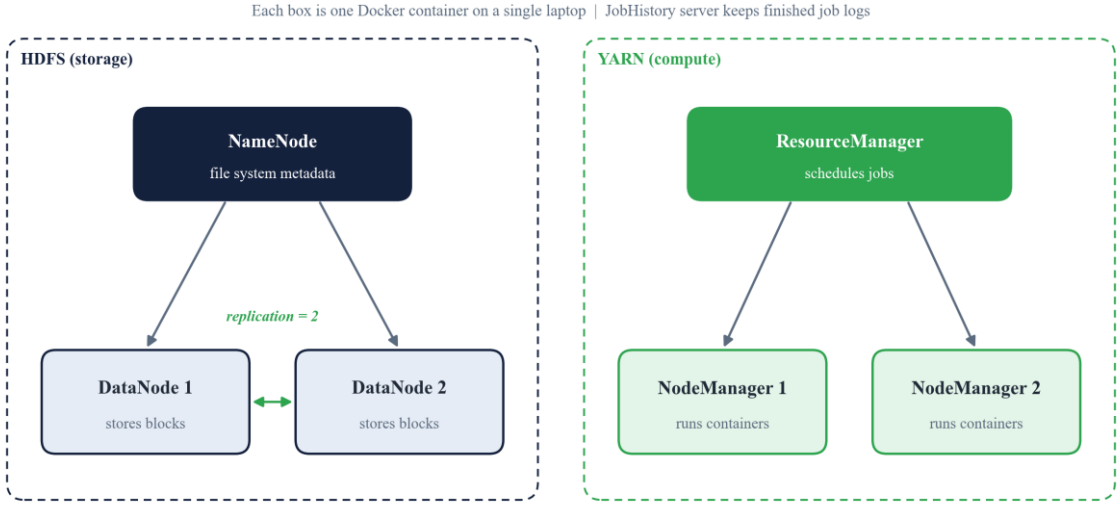


Figure 2.2: HDFS and YARN cluster layout

Memory-Aware Stages

Running all 23 services at once needs far more memory than a laptop has. The services are therefore grouped into stages (Table 2.1). The script scripts/stage.sh starts exactly the services

a stage needs and stops every other running service, so the laptop never runs more than one stage at a time. The full stack works with 16 GB of RAM, and a LITE mode with one NodeManager covers 8 GB machines.

Table 2.1: Memory-aware stages

Stage	Services running	Used for
core	PostgreSQL, NameNode, 2 DataNodes, ResourceManager, 2 NodeManagers, JobHistory	HDFS and YARN demos
batch	core + edge client	Download, Pig, MapReduce, Sqoop
warehouse	core + Hive metastore + HiveServer2 + edge	Hive, Sqoop import and export
ml	core + Hive metastore + edge	Spark, Mahout
serving	core + ZooKeeper + HBase + edge	HBase loads and lookups
impala	HDFS + metastores + Impala daemons (YARN stopped)	Impala queries
streaming	HDFS + ZooKeeper + HBase + Kafka + Storm (YARN stopped)	Live topology
orchestrate	core + Airflow	DAG runs (Airflow switches stages per task)

2.3 Data Pre-Processing Steps

Figure 2.3 shows the daily batch pipeline. Its first half prepares the raw data for analysis:

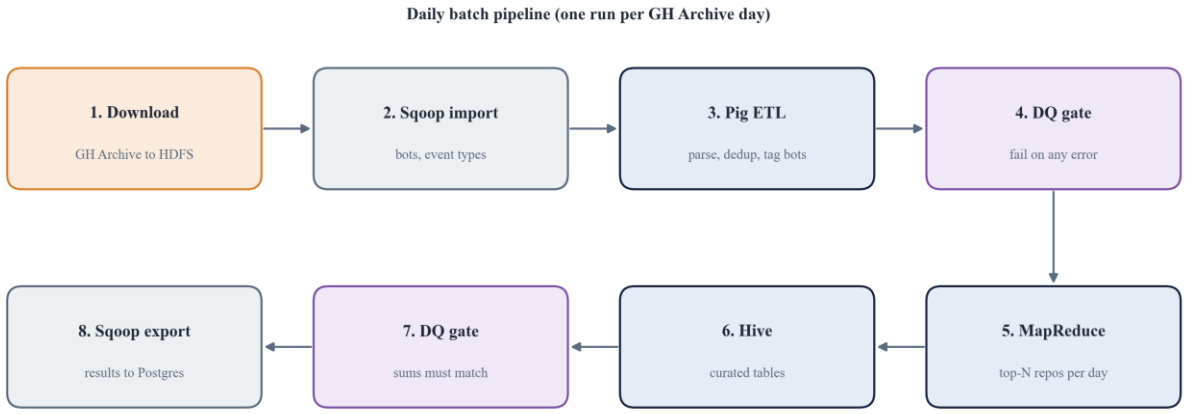


Figure 2.3: Daily batch pipeline

- **Download and load:** scripts/download_gharchive.sh fetches the hourly .json.gz files for the chosen dates and hours, checks them and loads them into the raw zone of HDFS under /gh/raw/events/dt=YYYY-MM-DD. The number of lines in each file is recorded for the quality gate.

- **Lookup import with Sqoop:** The list of known bots and the list of event types are kept in PostgreSQL and imported into Hive with Sqoop, so that Pig and Hive can join against them.
- **Parsing with a Java UDF:** The Pig script pig/etl_events.pig reads every line as text and parses it with ParseGhEvent, a Java user-defined function. It extracts about thirty typed fields such as event id, type, actor, repository, time, pull request number and merge status. A line that cannot be parsed becomes null and is counted as malformed.
- **Deduplication:** GH Archive occasionally repeats an event. Pig groups the records by event_id and keeps exactly one row for each.
- **Bot tagging:** Each actor is tagged as a bot if the login ends in [bot], if it appears in the known-bots list (a replicated, map-side join), or if the name ends in -bot or _bot. The reason is stored in a bot_reason column.
- **Columnar storage:** The clean records are written as Parquet into gh.events_clean, partitioned by date and event type, which makes later queries read only the columns and partitions they need.
- **Data-quality gate:** dq/check_clean.sql compares the counts at every stage. It requires the downloaded lines to equal the lines Pig read, the clean rows to equal Pig's output, no duplicate ids, no null keys, only known event types, dates that match the timestamps, and fewer than 0.1% malformed lines. Hive's assert_true() fails the query, and the Airflow task, if any rule is broken.

2.4 Main Processing Steps

Batch analytics

- **MapReduce top-N:** TopReposByTypeDay is a hand-written Java MapReduce job. The mapper reads the Parquet files and emits a composite key of (day, event type, repository) with a count of 1. A combiner sums counts locally, a custom partitioner sends every repository of one (day, type) group to the same reducer, and the reducer keeps a min-heap of size N to output the top N repositories.
- **Hive curated tables:** hive/curated/daily.sql builds daily tables for trending repositories, language trends (joined with the catalogue), the activity clock, the bot share and maintainer health. hive/curated/window.sql builds tables across the whole loaded window, such as the trending window and repository health. A second quality gate checks that every curated table adds back up to the clean count for its day.
- **Sqoop export:** Curated results are exported from Hive back to PostgreSQL, where the dashboard can read them directly.

- **Impala:** Impala shares the Hive metastore and reads the same Parquet files. Because its C++ daemons are always running, it answers dashboard queries in about one to two seconds instead of tens of seconds.

Machine learning

- **Feature engineering (Spark):** spark/features.py builds per-repository features (activity mix, contributor counts, Gini coefficient and top contributor share) and, for every new repository, its activity in the first F hours and the human stars in the following L hours.
- **Viral prediction (Spark MLlib):** A repository is labelled viral when its stars in the label window reach the 97th percentile, so roughly the top 3% of new repositories are positive. The data is split by time, and logistic regression is compared with gradient-boosted trees using AUC-ROC, AUC-PR and lift.
- **Recommendations (Mahout):** Each user's stars form an implicit, boolean rating. Mahout's item-based collaborative filtering with log-likelihood ratio similarity finds the 20 most similar repositories for each repository and the top 10 recommendations for each user. Spark ALS and a popularity baseline are trained on the same split for comparison, holding out each user's latest 20% of stars.
- **Clustering (Mahout k-means):** Repositories are clustered by their activity profile with k = 6 to find groups such as solo push projects and discussion-heavy projects.
- **Serving (HBase):** Results are shaped into HBase row keys with a two-character MD5 salt, for example <salt>|<repo_id>, so that new writes are spread across regions instead of all hitting the last one. They are bulk loaded with ImportTsv into the tables repo_hourly, repo_similar and user_recs.

Real-time stream

The speed layer is shown in Figure 2.4. A producer replays a real GH Archive hour into the Kafka topic gh.events at an adjustable speed (60 times faster by default). The Storm topology parses each event and sends it to three windowed bolts that use event time: TypeMinuteBolt counts events per minute by type with a bot split, RepoWindowCountBolt and a ranker keep a rolling 5-minute top 10 of repositories, and SpikeBolt keeps an exponentially weighted mean and variance per repository and raises an alert when a minute's count is at least 15 and more than 4 standard deviations above the mean. Results go to the HBase tables live_minute, live_topk and spike_alerts.

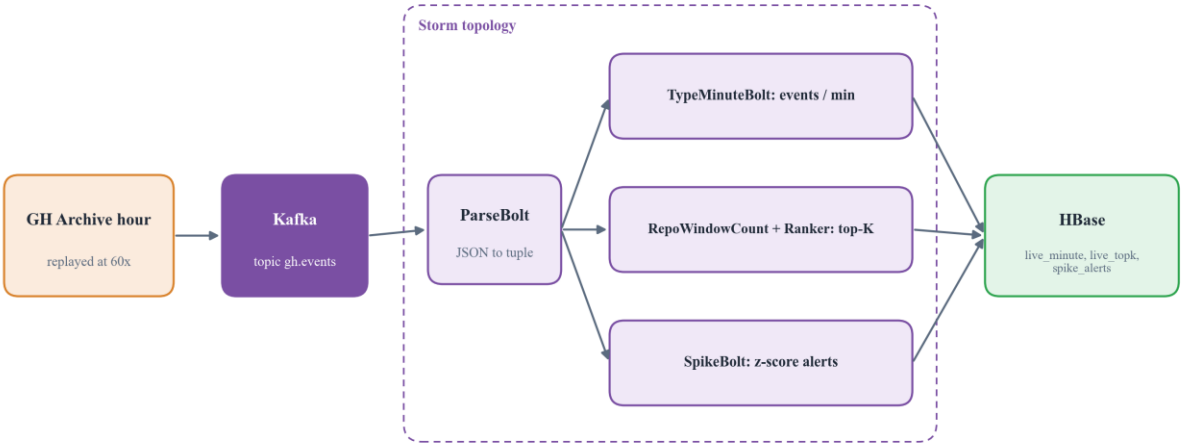


Figure 2.4: Real-time path: Kafka to Storm to HBase

Orchestration

Apache Airflow runs the batch steps as two DAGs (Figure 2.5). gh_pulse_daily runs once for each day of data, from download to the curated quality gate. gh_pulse_models runs once the window is loaded: it seeds the catalogue from the GitHub API, imports it with Sqoop, builds the window tables, runs the three machine learning branches in parallel, loads HBase and exports the results. Each task retries with exponential backoff, except the quality gates, which must stop and not retry.

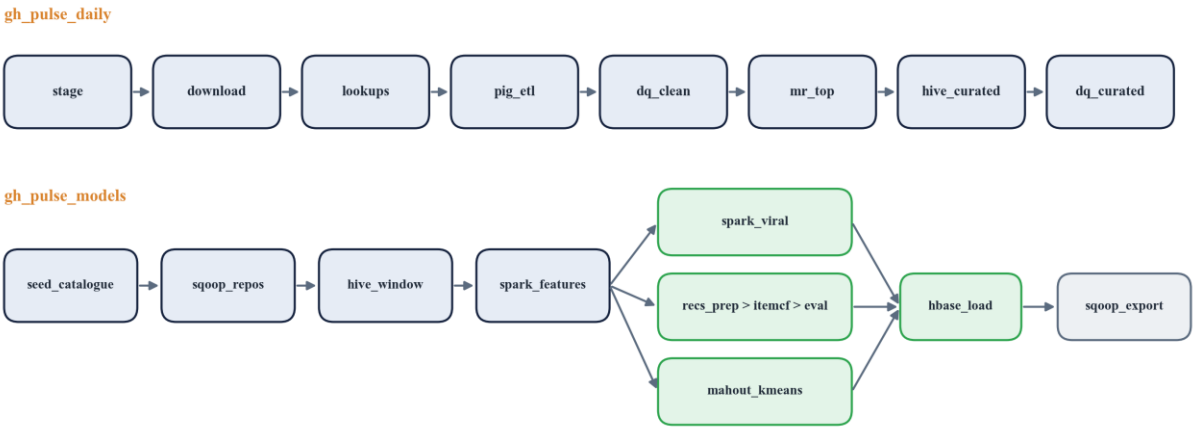


Figure 2.5: The two Airflow DAGs

2.5 Tools Used

Table 2.2 lists every tool in the stack, the version used and the single job it does in GitPulse.

Table 2.2: Tools used and their roles

Tool	Version	Role in GitPulse
HDFS	Hadoop 3.3.6	Distributed storage for raw, clean, curated and ML zones (replication 2)
YARN	Hadoop 3.3.6	Allocates CPU and memory to MapReduce, Pig, Hive and Spark jobs

Tool	Version	Role in GitPulse
MapReduce	Hadoop 3.3.6	Custom Java job for the daily top-N repositories per event type
Apache Pig	0.18.0	ETL: parse JSON with a Java UDF, deduplicate, tag bots, write Parquet
Apache Hive	3.1.3	SQL data warehouse for curated tables and data-quality gates
Apache Sqoop	1.4.7	Import lookups and catalogue from PostgreSQL; export results back
Apache Spark	3.5.9	Feature engineering, viral model (MLlib), ALS recommendations
Apache Mahout	0.13.0	Item-based collaborative filtering and k-means clustering
Apache HBase	2.5.16	Low-latency serving of similar repos, recommendations and live counters
Apache Kafka	3.9 (Confluent 7.9)	Durable message queue for the event stream
Apache Storm	2.8.9	Real-time topology: per-minute counts, top-K, spike alerts
Apache Impala	4.5.3	Interactive SQL for the dashboard on the same Parquet tables
Apache Airflow	3.3.2	Orchestrates both pipelines as DAGs with retries
ZooKeeper	3.8	Coordination for HBase and Kafka
PostgreSQL	16	Repository catalogue, Hive metastore and exported results
Docker Compose	Docker Desktop	Runs every node as a container on one laptop
Tableau	Desktop / Public	Dashboard over Impala and PostgreSQL extracts

2.6 Software and Hardware Requirements

Table 2.3: Software and hardware requirements

Component	Requirement
Processor	Quad-core 64-bit CPU minimum; 8 or more cores recommended (x86_64 or Apple Silicon)
RAM	16 GB recommended for the full stages; 8 GB works with LITE=1 mode
Storage	About 30 GB free (15 GB for the sample and demo datasets, 60 GB for the large dataset)
GPU	Not required
Operating System	Windows 10/11, macOS or Linux
Container runtime	Docker Desktop with Docker Compose
Java	Java 8 inside the containers; Maven 3.9 with JDK 17 in a build container
Python	Python 3.12 (Spark jobs, catalogue seeding, Kafka producer, tests)
Shell	Bash (Git Bash on Windows)

Component	Requirement
Network	Internet access to download GH Archive files and call the GitHub API
Optional	GitHub personal access token (5,000 API requests per hour instead of 60)
Test machine used	Windows 11, 16 CPU threads, 32 GB RAM (15.5 GB given to Docker)

2.7 Dataset Used

Event data comes from GH Archive for September 2025, before GitHub trimmed public event payloads in October 2025. Repository metadata comes from the GitHub REST API and is cached on disk for seven days, with stale entries revalidated by ETag so that they do not use quota. Three dataset sizes are supported with the same commands: sample (3 hours), demo (2 days) and large (8 days). The results in this report use the 15 hours listed in Table 2.4.

Table 2.4: Hours of GH Archive data used

Dataset	Hours (UTC)	Note
sample	2025-09-01, 12:00 to 14:00	A Monday
medium	2025-09-07 and 2025-09-08, 10:00 to 15:00 each	A Sunday and a Monday

Table 2.5: Dataset metrics

Metric	Value
Total events (15 hours)	2,397,993
Events per hour	About 157,000 to 167,000
Clean, deduplicated events	2,397,992
Human-only events	1,685,763
Curated repository-days	814,035
Curated human stars	76,864
New repositories for the viral model	67,585
Users in the recommendation evaluation	205

CHAPTER 3

Results

3.1 Results

Data quality

Every record is accounted for at every stage (Figure 3.1 and Table 3.1). Out of 2,397,993 downloaded lines, none were malformed and exactly one duplicate event id was removed. Every curated table sums back to the clean count for its day. During development the gates also caught two real problems: an hourly file with no trailing newline, which made the line count differ from Pig's count by one, and 140 events on 2025-09-08 that were already in GitHub's newer trimmed format.

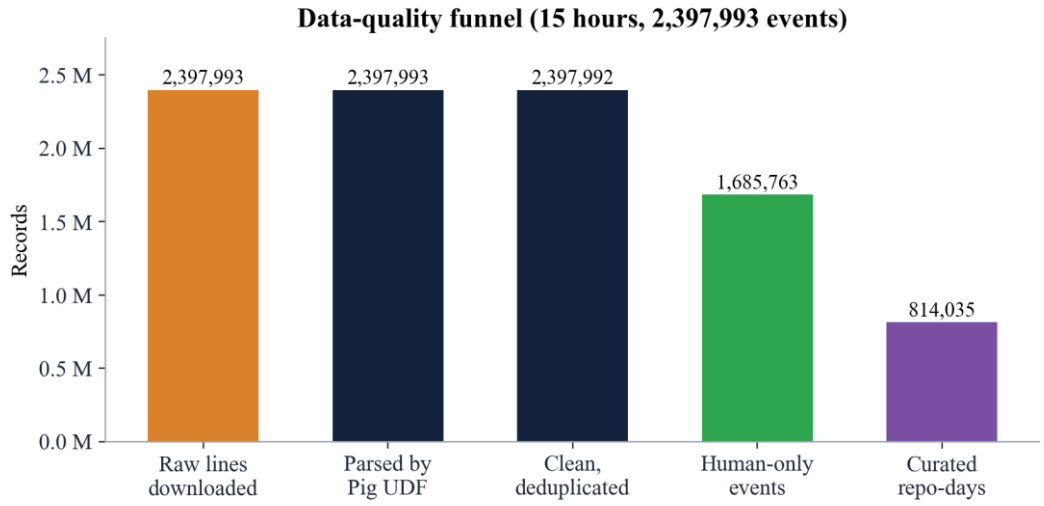


Figure 3.1: Data-quality funnel

Table 3.1: Record counts through the pipeline

Stage	Records
Raw lines downloaded	2,397,993
Parsed by the Pig UDF	2,397,993 (0 malformed)
Clean, deduplicated	2,397,992 (1 duplicate removed)
Clean, human only	1,685,763
Curated repository-days	814,035
Curated human stars	76,864

Answers to the analytical questions

- **Q1 Trending:** By human stars on 2025-09-01 between 12:00 and 15:00 UTC, the leaders were dockur/windows (168 stars), pewdiepie-archdaemon/dionysus (93) and JetBrains/koog (56). Raw push volume, in contrast, was led by an automated repository

with 4,138 pushes in three hours, which is why the catalogue is ranked by distinct human actors.

- **Q3 Activity clock:** Sunday's 10:00 to 15:00 UTC hours carried about 157,000 events per hour against 162,000 on the weekday, only about 3% less.
- **Q4 Bots:** Bots produce 28% to 32% of all events. They write 60% of commit comments, 52% of issue comments and 49% of pull request events (Figure 3.2).
- **Q5 Maintenance:** For human-authored pull requests, NixOS/nixpkgs merges 93% of closed pull requests with a median of about 38 hours open.
- **Q6 Solo versus community:** Of 11,754 repositories with at least 20 events, 6,083 are solo projects, 1,877 are small teams and only 407 are community projects (top contributor below 50% and at least 10 contributors), as shown in Figure 3.3.
- **Q10 Clusters:** Mahout k-means with k = 6 separated push-only solo projects, bot-driven pushes, pull request and push collaboration, discussion-heavy projects, and star and issue magnets.

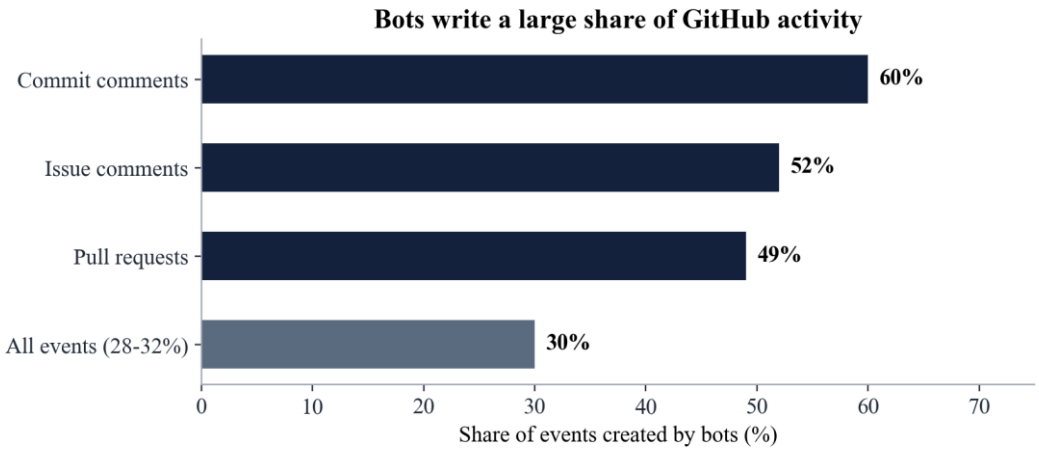


Figure 3.2: Share of events created by bots

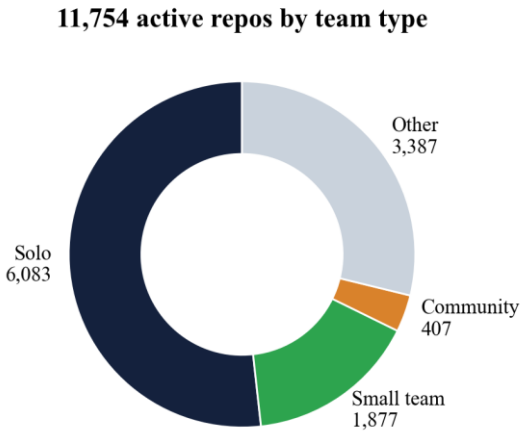


Figure 3.3: Active repositories by team type

Machine learning results

Table 3.2 and Figure 3.4 compare the recommenders on 205 users, holding out each user's latest 20% of stars. Mahout item-based filtering is best on every metric. Its recall at 10 is about six times that of the popularity baseline, and it covers 53% of the catalogue against 24% for Spark ALS.

Table 3.2: Recommendation results

Method	Hit rate	Recall@10	Precision@10	Coverage
Mahout item CF (LLR)	0.263	0.251	0.028	0.53
Spark ALS (implicit, rank 16)	0.259	0.237	0.026	0.24
Popularity baseline	0.044	0.041	0.004	0.01

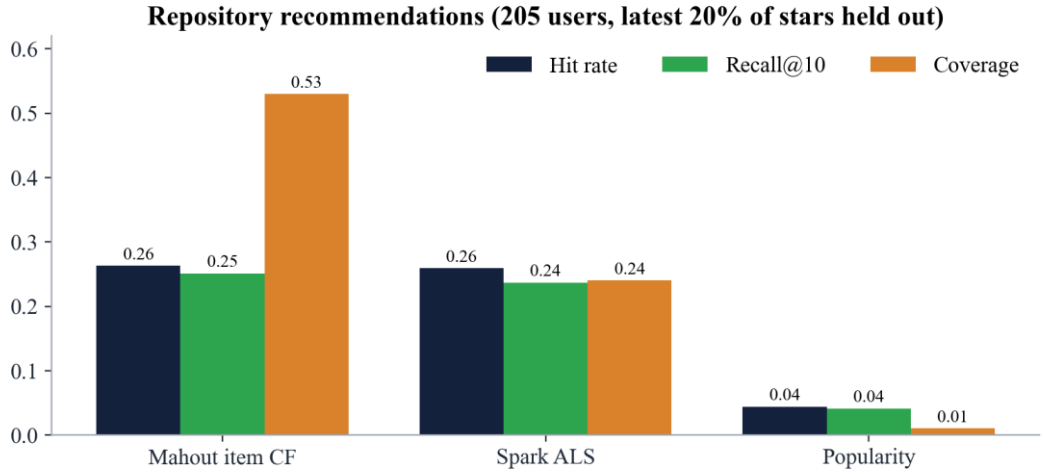


Figure 3.4: Comparison of recommendation methods

For viral prediction (1 hour of features, 3 hour label, split by time, 67,585 new repositories and 40 positives in the test set), logistic regression reached an AUC-ROC of 0.73 and an AUC-PR

of 0.0033 against a base rate of 0.0012, a lift of 2.7 times. Gradient-boosted trees reached an AUC-ROC of 0.55. This is a weak but real signal, and the model is designed for the longer 24 hour and 5 day setting on the large dataset.

Table 3.3: Viral prediction results

Model	AUC-ROC	AUC-PR	Lift over base rate
Logistic regression	0.73	0.0033	2.7x
Gradient-boosted trees	0.55	n/a	n/a

Real-time results

One GitHub hour of 166,000 to 167,000 events was replayed at 60 times speed in about 65 seconds through Kafka and Storm with 0 failed tuples. It produced 60 per-minute rows, 60 rolling top 10 windows and several spike alerts. For example, ursacomputing/crossbow went from no activity to 104 events in one minute, and hotspotlab/hourly reached 18 events against a mean of 0.41 (z = 12.8).

Engine benchmark

The same three queries were run on the same Parquet tables with MapReduce, Pig, Hive on MapReduce, Spark SQL and Impala (Table 3.4 and Figure 3.5). At this data size start-up time dominates: Impala's long-running daemons answer in about 1.3 seconds, while the JVM and YARN engines need 20 to 40 seconds before they read any data. MapReduce scales worst because it starts one map task for each small Parquet file created by the date and event-type partitions, while Pig combines small files into larger splits. A second worker gave a speed-up of only 1.0 to 1.9 times, because both NodeManagers share the same laptop CPU.

Table 3.4: Median query time in seconds by engine

Size (clean Parquet)	Workers	MapReduce	Pig	Hive (MR)	Spark SQL	Impala
sample (0.028 GB)	1	62 to 71	27 to 43	20 to 39	37 to 38	1.2 to 1.3
sample (0.028 GB)	2	53 to 59	27 to 38	15 to 28	20 to 37	n/a
medium (0.130 GB)	1	162 to 175	40 to 48	23 to 39	38 to 39	1.4 to 1.5

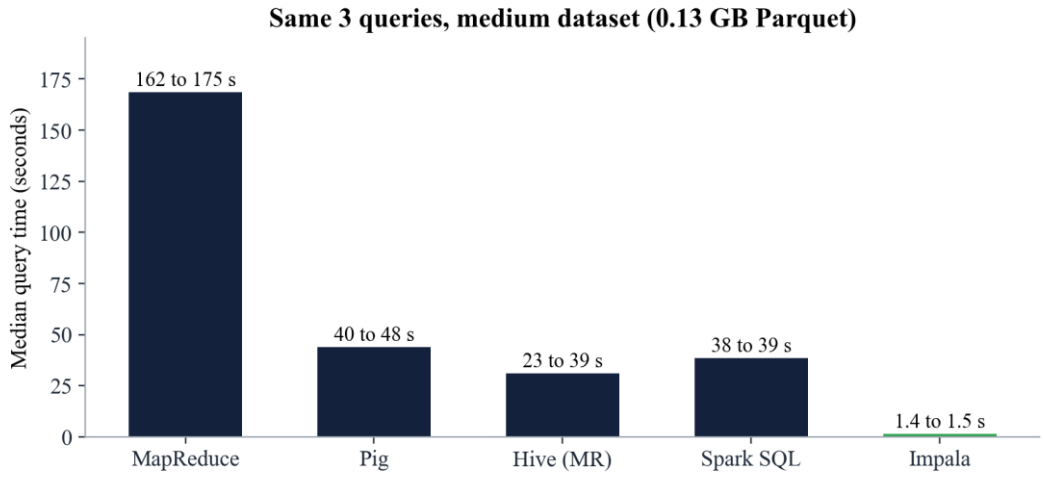


Figure 3.5: Median query time by engine, medium dataset

End-to-end test run

To test the complete system we ran it from scratch on a Windows 11 laptop with 15.5 GB of memory given to Docker, using the sample dataset (2025-09-01, 12:00 to 14:00 UTC). The cluster smoke test confirmed 2 live DataNodes, 2 NodeManagers, a completed MapReduce job and an average block replication of 2.0. All 10 JUnit tests (Pig UDF, MapReduce on LocalJobRunner and the Storm topology on LocalCluster) and the pytest suite for the Pig ETL passed. All 25 steps of both pipelines then passed, in about 58 minutes in total (Table 3.5 and Figure 3.6).

The run loaded 497,262 events with 0 malformed lines and 0 duplicates. The bot share was 31.6%, inside the 28% to 32% range seen on the larger data. The final end-to-end script found 375 trending rows exported to PostgreSQL, a working HBase similar-repositories lookup (people who starred vinta/awesome-python also starred project-based-learning, 100-Days-Of-ML-Code and sindresorhus/awesome), and 10,614 PushEvents counted by Storm in HBase from a two-minute replay.

Table 3.5: Steps of the end-to-end test run

#	Pipeline step	Time (s)	Result
1	stage_warehouse	47	Passed
2	ddl	26	Passed
3	catalogue_init	1	Passed
4	sqoop_import_lookups	93	Passed
5	pig_etl	178	Passed
6	dq_clean	103	Passed
7	mr_top	105	Passed
8	hive_curated	551	Passed

#	Pipeline step	Time (s)	Result
9	dq_curated	191	Passed
10	seed_catalogue	134	Passed
11	sqoop_import_repos	65	Passed
12	hive_window	278	Passed
13	stage_ml	13	Passed
14	spark_features	68	Passed
15	spark_viral	95	Passed
16	recs_prep	53	Passed
17	mahout_itemcf	490	Passed
18	recs_eval	97	Passed
19	mahout_kmeans	260	Passed
20	hbase_prep	57	Passed
21	stage_serving	48	Passed
22	hbase_init	22	Passed
23	hbase_load	352	Passed
24	stage_warehouse (again)	41	Passed
25	sqoop_export	114	Passed

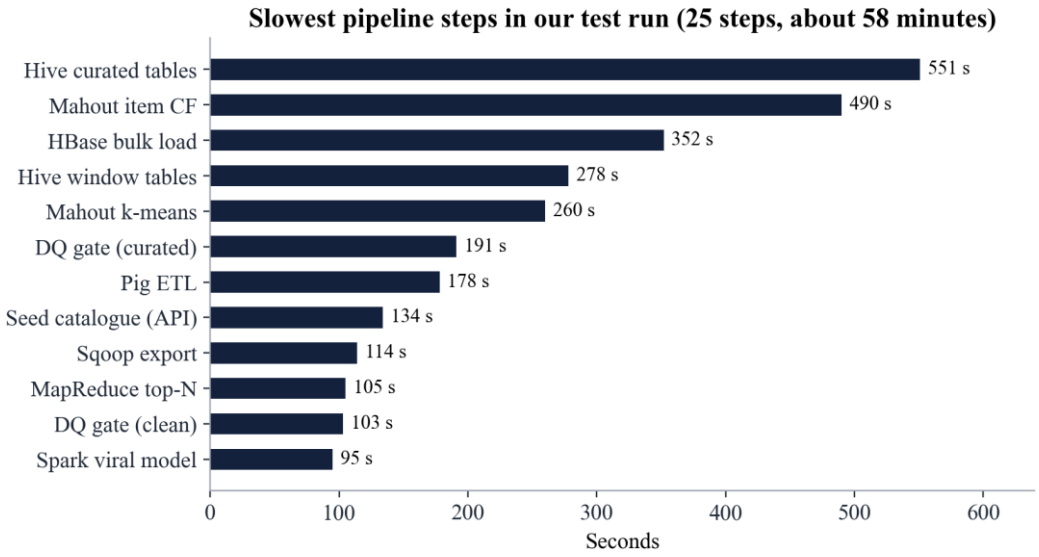


Figure 3.6: Slowest steps of the test run

Table 3.6: Summary of tests

Check	Outcome
Cluster smoke test	2 DataNodes, 2 NodeManagers, MapReduce pi job, replication 2.0
JUnit (Pig UDF, MapReduce, Storm)	10 of 10 passed
pytest (Pig ETL in local mode)	1 of 1 passed
Data-quality gates (clean and curated)	All assertions passed

Check	Outcome
Batch and models pipelines	25 of 25 steps passed
End-to-end checks (PostgreSQL, HBase, Storm)	Passed

Cluster user interfaces

Figures 3.7 to 3.10 show the web interfaces of the running cluster: the HDFS NameNode with both DataNodes in service, the MapReduce JobHistory with completed Hive jobs, Impala's completed queries and the HBase Master with its region server.

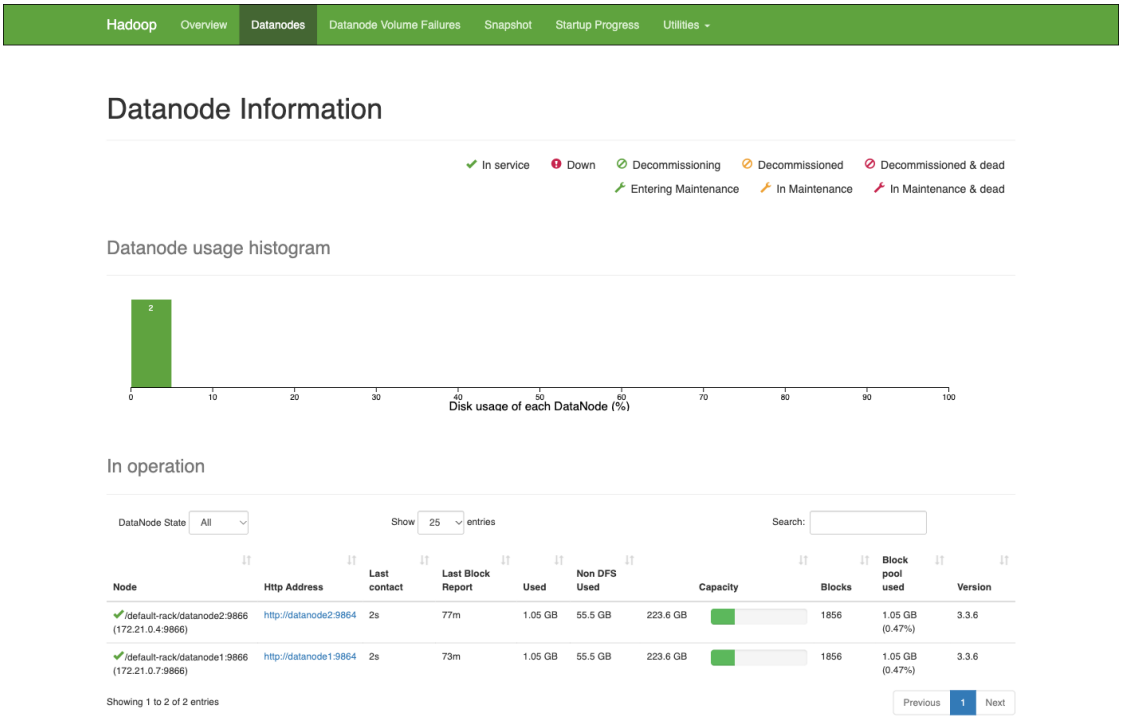


Figure 3.7: HDFS NameNode: two DataNodes in service



Figure 3.8: MapReduce JobHistory: completed jobs

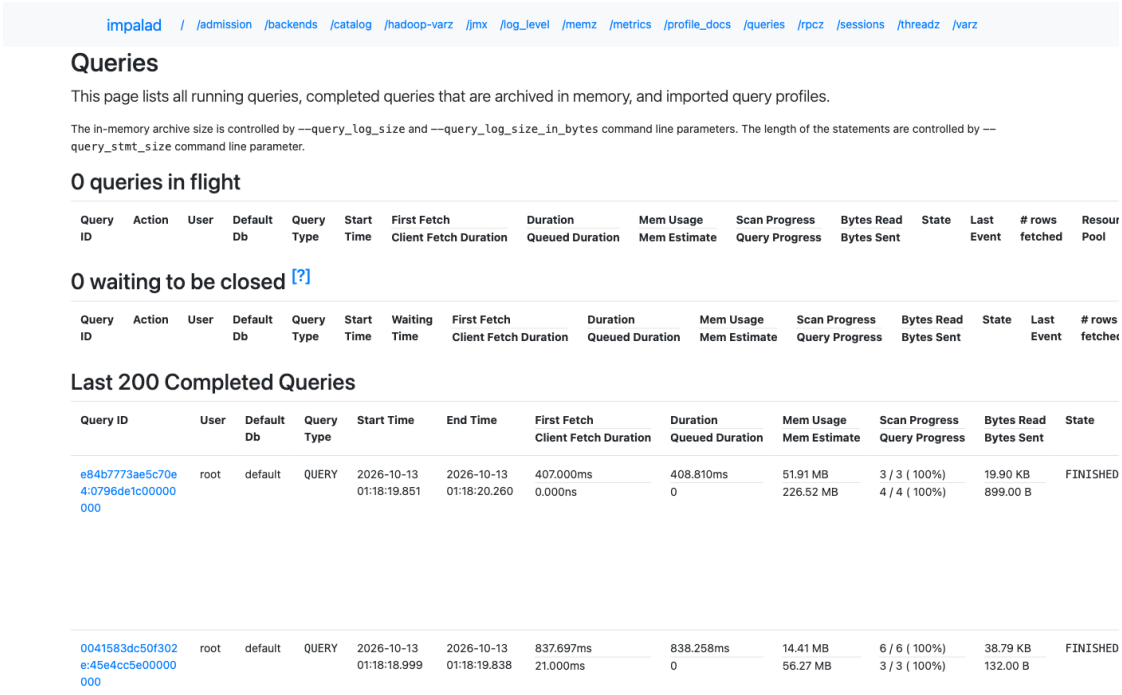


Figure 3.9: Impala daemon: completed queries

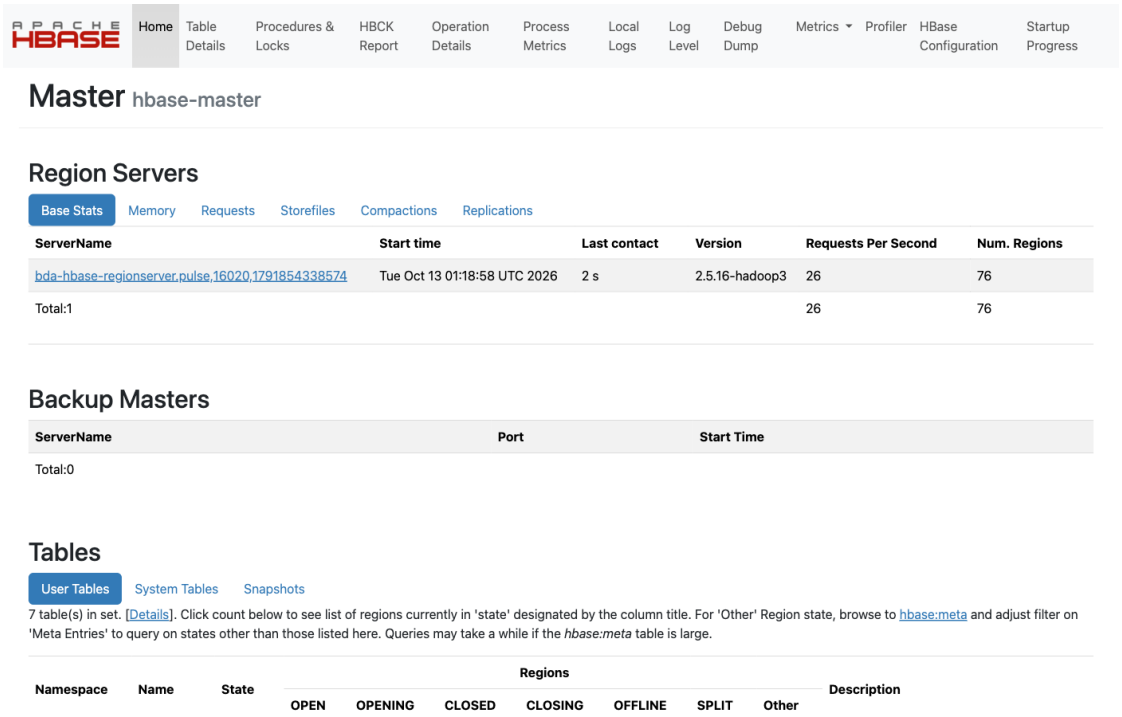


Figure 3.10: HBase Master with its region server

CHAPTER 4

Conclusion and future work

This project delivers GitPulse, a complete and tested big data analytics platform for public GitHub activity. It combines twelve tools from the Hadoop ecosystem, each used for the job it suits best: Pig for ETL, MapReduce for top-N ranking, Hive and Impala for SQL analytics, Spark and Mahout for machine learning, HBase for low-latency serving, Kafka and Storm for real-time processing, Sqoop for moving data to and from PostgreSQL, and Airflow for orchestration. Memory-aware stages make the whole multi-node cluster run on a single laptop.

The results show that the pipeline is trustworthy and that the data holds real insight. Data-quality gates account for every one of the 2.4 million events, with no malformed lines and one duplicate. Bots create close to a third of all GitHub activity and more than half of all comments. Most active repositories are solo projects. Mahout item-based filtering recommends repositories about six times better than popularity. The real-time path processes a full GitHub hour in about a minute with no failed tuples. The engine benchmark shows that for small, partitioned data, start-up cost and the small-files problem matter more than raw processing speed, which is why Impala answers in about a second while the YARN engines take 20 to 170 seconds. An end-to-end run on a Windows laptop passed all 25 pipeline steps and every test.

Future enhancements include running both pipelines on the full 8-day dataset, which the viral model is designed for, and replacing the replay with the live GitHub API feed that is already implemented. Other improvements are comparing Storm with Spark Structured Streaming, deploying the stack on a real multi-node cluster or a managed cloud service to measure true horizontal scaling, compacting small Parquet files to remove the small-files penalty, and publishing the Tableau dashboard for public use.

REFERENCES

[1] GH Archive, "GH Archive: A Project to Record the Public GitHub Timeline." Available: https://www.gharchive.org/.

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2004.

[3] K. Shvachko, H. Kuang, S. Radia and R. Chansler, "The Hadoop Distributed File System," in Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010.

[4] V. K. Vavilapalli et al., "Apache Hadoop YARN: Yet Another Resource Negotiator," in Proc. 4th ACM Symposium on Cloud Computing (SoCC), 2013.

[5] C. Olston, B. Reed, U. Srivastava, R. Kumar and A. Tomkins, "Pig Latin: A Not-So-Foreign Language for Data Processing," in Proc. ACM SIGMOD, 2008.

[6] A. Thusoo et al., "Hive: A Warehousing Solution over a Map-Reduce Framework," Proceedings of the VLDB Endowment, vol. 2, no. 2, 2009.

[7] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in Proc. 9th USENIX NSDI, 2012.

[8] J. Kreps, N. Narkhede and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in Proc. NetDB Workshop, 2011.

[9] A. Toshniwal et al., "Storm@Twitter," in Proc. ACM SIGMOD, 2014.

[10] M. Kornacker et al., "Impala: A Modern, Open-Source SQL Engine for Hadoop," in Proc. 7th Biennial Conference on Innovative Data Systems Research (CIDR), 2015.

[11] F. Chang et al., "Bigtable: A Distributed Storage System for Structured Data," in Proc. 7th USENIX OSDI, 2006.

[12] B. Sarwar, G. Karypis, J. Konstan and J. Riedl, "Item-Based Collaborative Filtering Recommendation Algorithms," in Proc. 10th International World Wide Web Conference (WWW), 2001.

[13] Y. Hu, Y. Koren and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008.

[14] T. Dunning, "Accurate Methods for the Statistics of Surprise and Coincidence," Computational Linguistics, vol. 19, no. 1, pp. 61-74, 1993.

[15] N. Marz and J. Warren, Big Data: Principles and Best Practices of Scalable Realtime Data Systems. Manning Publications, 2015.